

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



XÂY DỰNG VÀ TÍNH CHỈNH PHIÊN BẢN TỐI GIẢN CỦA MÔ HÌNH LLAMA 2

BÁO CÁO BÀI TẬP
Học phần: AIT3009# 1

Sinh viên thực hiện

Nguyễn Thị Thanh Huyền - 23020381

Nguyễn Thị Minh Ly - 23020399

Đặng Minh Nguyệt - 23020407

Giảng viên hướng dẫn

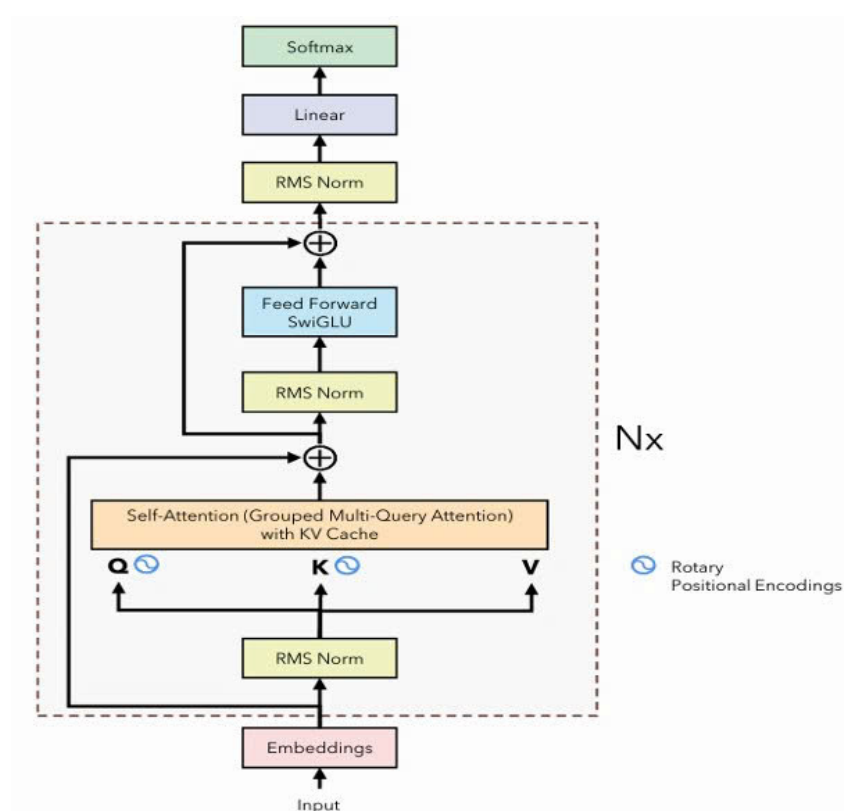
ThS. Ngô Minh Hương

HÀ NỘI - 2026

1 Kiến trúc LLAMA-2

1.1 Kiến trúc tổng quát

Llama 2 là mô hình ngôn ngữ lớn dựa trên kiến trúc Transformer dạng decoder-only. Mô hình nhận chuỗi token đầu vào, xử lý qua các lớp Transformer decoder với cơ chế causal self-attention, và dự đoán token tiếp theo thông qua lớp tuyến tính kết hợp softmax. Mỗi decoder layer tuân theo thiết kế pre-norm, với luồng xử lý: RMSNorm $\rightarrow$ Causal Self-Attention $\rightarrow$ Residual connection, tiếp theo là RMSNorm $\rightarrow$ Feed-Forward Network $\rightarrow$ Residual connection. Đặc biệt, cơ chế Rotary Positional Embeddings (RoPE) được tích hợp trực tiếp vào các vector query và key để mã hóa linh hoạt thông tin vị trí tương đối.



Hình 1: Kiến trúc Llama 2

1.2 Cài đặt

Cài đặt các thành phần chính của Llama-2 trong `llama.py`, `rope.py`, `optimizer.py`, `classifier.py` chi tiết như sau:

- `llama.Attention.forward`: Thực hiện self-attention bằng cách biến đổi đầu vào thành Q, K, V và tổ chức theo nhiều head để học các quan hệ khác nhau. Thông tin vị trí được tích hợp qua RoPE, trong khi GQA cho phép chia sẻ K và V giữa các head nhằm tối ưu bộ nhớ. Kết quả được tổng hợp và chiếu qua lớp tuyến tính để tạo đầu ra.

- `llama.RMSNorm.norm`: Thực hiện chuẩn hóa RMSNorm dựa trên giá trị trung bình bình phương của vector đầu vào, kết hợp với ϵ để đảm bảo ổn định số học. Phép chuẩn hóa này điều chỉnh độ lớn của vector nhưng vẫn giữ nguyên hướng trong không gian biểu diễn.
- `llama.Llama.forward`: Kết hợp embedding, các decoder layer và RMSNorm để xây dựng biểu diễn của chuỗi. Biểu diễn này được chiếu sang không gian từ vựng để tạo logits; khi suy luận chỉ tính tại vị trí cuối nhằm tối ưu hiệu năng.
- `llama.Llama.generate`: Sinh văn bản theo cơ chế tự hồi quy, trong đó chuỗi hiện tại được đưa trở lại mô hình để dự đoán token tiếp theo với temperature sampling. Chuỗi được giới hạn theo độ dài tối đa và các token được nối dần để tạo kết quả.
- `rope.apply_rotary_emb`: Áp dụng RoPE bằng cách biểu diễn Q và K dưới dạng cặp và thực hiện phép quay phụ thuộc vị trí thông qua các hàm sin-cos, từ đó giúp tích hợp thông tin vị trí trực tiếp vào attention và bảo toàn quan hệ tương đối giữa các token.
- `optimizer.AdamW.step`: Cập nhật tham số bằng cách duy trì trung bình động bậc nhất và bậc hai của gradient, kèm hiệu chỉnh bias để ổn định quá trình học. Tham số được cập nhật với gradient đã chuẩn hóa, sau đó áp dụng weight decay tách biệt.
- `classifier.LlamaEmbeddingClassifier.forward`: Sử dụng biểu diễn ẩn tại token cuối làm đặc trưng đại diện cho chuỗi, sau đó được điều chỉnh qua dropout, đưa vào lớp phân loại và log-softmax để thu được phân phối xác suất trên các lớp.

```

PS C:\23020407\ASM1-Development-LLM-model> .\venv\Scripts\python.exe rope_test.py
Rotary embedding test passed!
PS C:\23020407\ASM1-Development-LLM-model> .\venv\Scripts\python.exe optimizer_test.py
Optimizer test passed!
PS C:\23020407\ASM1-Development-LLM-model> .\venv\Scripts\python.exe sanity_check.py
Your Llama implementation is correct!

```

Hình 2: Kết quả chạy 03 test cài đặt

2 Đánh giá mô hình

2.1 Sinh phân tiếp nối văn bản

Yêu cầu: Bắt đầu với câu: *"I have wanted to see this thriller for a while, and it didn't disappoint. Keanu Reeves, playing the hero John Wick, is".*

Nhận xét:

- Văn bản sinh ra xuất hiện nhiều câu vô nghĩa, không liên quan đến ngữ cảnh ban đầu. Nguyên nhân do mô hình được pretrained trên tập dữ liệu truyện thiếu nhi, dẫn đến văn phong kể chuyện đơn giản, phi logic so với bối cảnh phim hành động.

